Technical Architecture Report: Configurable Furnace Optimization Pipeline Framework

Link of Video→

Furnace_Pipeline_Standards.mp4

1. Executive Mandate: Systems Objective and Design Philosophy

In traditional industrial automation, optimization logic is frequently trapped within hardcoded scripts that are difficult to maintain and nearly impossible to scale. This architecture addresses the strategic necessity of moving toward a modular, furnace-agnostic optimization pipeline. By decoupling the underlying logic from specific furnace hardware, we significantly reduce technical debt and accelerate the deployment of optimization models across diverse industrial environments. This transition is not merely a software update; it is a shift from rigid, site-specific code to a flexible engine capable of adapting to any furnace configuration.

The framework is built upon four core architectural objectives:

- **Modular:** Components are engineered as independent functional blocks to allow for granular updates and maintenance.
- **Configurable:** System behavior is dictated entirely by external parameters (DB/CSV), eliminating the need for code changes during operational shifts.
- **Reusable:** Core engineering logic is abstracted to be applicable across different sites and furnace types.
- **Pipeline-based:** Data flows through a structured, sequential progression that enforces data integrity at every stage.

The "So What?": Strategic Impact This architecture transforms raw historian data into actionable recommendations while providing a critical safety net. By standardizing ingestion and processing, the system prevents "production-stop" events caused by hardcoded logic failures during furnace changeovers or tag re-addressing. This framework ensures that optimization engines receive high-quality, validated inputs, leading to reliable and repeatable furnace performance enhancements.

The following sections detail a logical architecture following a **Data Maturity Model**, progressing from raw data to optimized setpoints.

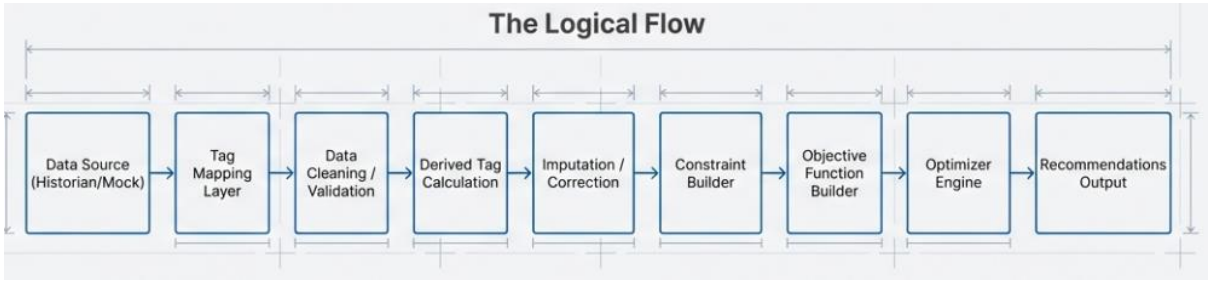
---

2. The Logical Architecture: End-to-End Pipeline Flow

The logical architecture views data processing as a linear progression of maturity. Information enters as raw, fragmented historian tags and is systematically refined through specialized layers until it emerges as a set of optimized operational recommendations.

Visual Flow Diagram

One sample example below



Pipeline Segmentation and Strategic Purpose

Pipeline	Strategic Purpose
Ingestion Pipeline	Load raw historian (PI) or mock data into the system environment.
Preprocessing Pipeline	Perform initial cleaning and schema validation of incoming data.
Feature Pipeline	Calculate inferred/calculated tags and evaluate complex formulas.
Imputation Pipeline	Address and fix "bad" or missing values to ensure data continuity.
Constraint Pipeline	Build the operational boundaries and constraints for the optimizer.
Objective Pipeline	Define the mathematical objective (e.g., fuel efficiency, heat retention).
Optimization Pipeline	Execute the optimizer engine to find the ideal furnace setpoints.
Reporting Pipeline	Format and output final recommendations for operational use.

This segmentation enables a "Plug-and-Play" capability. By isolating these functions, the system can adapt to different furnace types simply by swapping configurations within specific segments without rewriting the core execution logic.

---

3. Orchestration Layer: Kedro-Based Pipeline Design

Kedro is utilized as the primary orchestration engine to manage production-grade complexity through a structured hierarchy. This ensures the transition from static scripts to a maintainable software product.

System Hierarchy

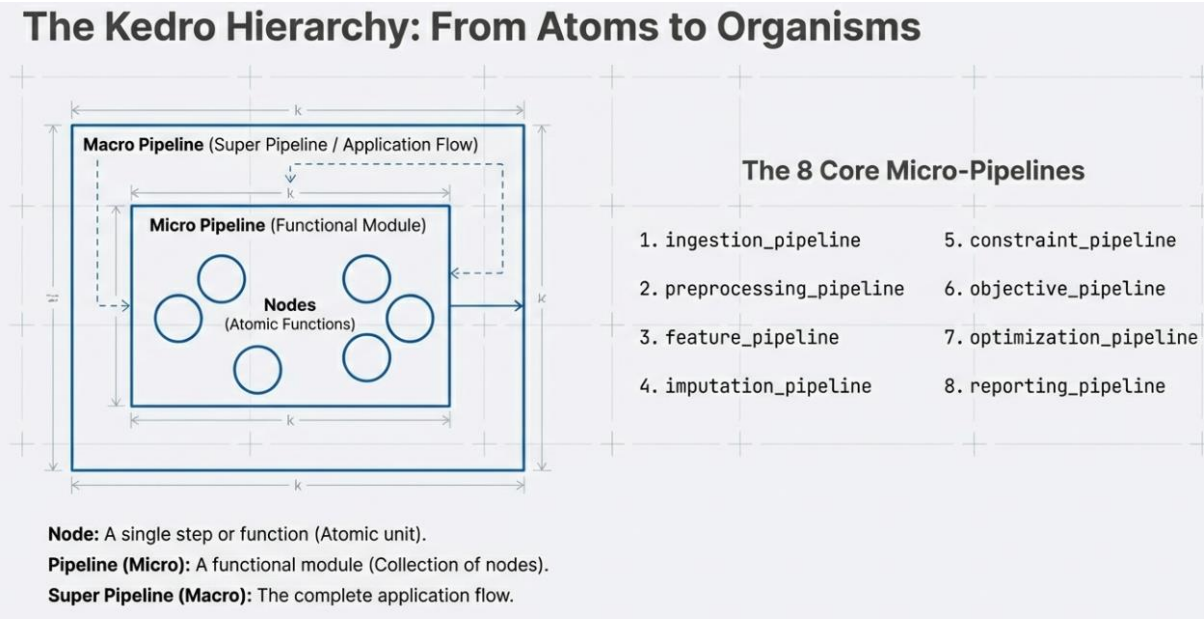
- **Node → Function:** The smallest unit of execution; a single Python function performing one task.
- **Pipeline → Module:** A collection of nodes forming a logical functional block (e.g., Preprocessing).
- **Super Pipeline → Application Flow:** The overarching orchestration of multiple pipelines into a complete furnace optimization application, executed via a centralized `runner.py`.

Pipeline Structure Example: `preprocessing_pipeline`

```
from kedro.pipeline import Pipeline, node

def create_preprocessing_pipeline():
    return Pipeline([
        node(
            func=remove_outliers,
            inputs="raw_data",
            outputs="filtered_data",
            name="remove_outliers_node"
        ),
        node(
            func=validate_schema,
            inputs="filtered_data",
            outputs="validated_data",
            name="validate_schema_node"
        ),
        node(
            func=normalize_units,
            inputs="validated_data",
            outputs="normalized_data",
            name="normalize_units_node"
        ),
    ])
```

Pro Tip: Conditional Execution The orchestration layer supports **conditional pipelines**. For example, the `imputation_pipeline` can be skipped entirely if the global configuration for a specific furnace id specifies `impute: False`.



---

4. Configuration-Driven Strategy: The "Everything Configurable" Principle

The primary engineering mandate is the total decoupling of logic from configuration. By moving parameters to external sources, we allow non-developers to adjust system behavior without requiring a code redeploy.

The MUST Principle

Everything must be configurable; nothing can be hardcoded. Ignoring this rule necessitates a full redeploy for every limit change or tag addition, creating an unscalable maintenance burden and increasing the risk of runtime failures.

Tag Configuration Schema

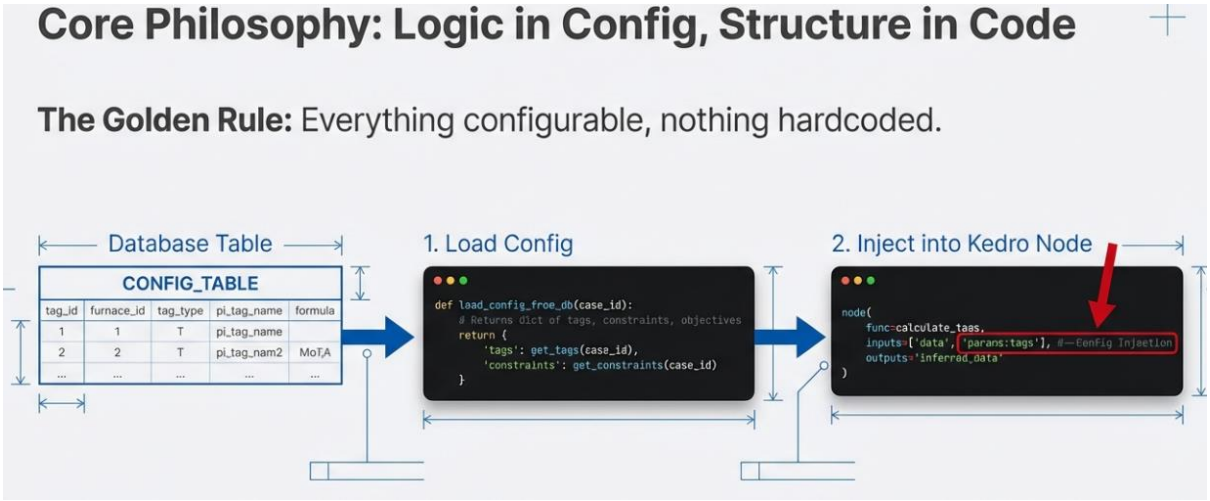
tag_id	furnace_id	tag_name	pi_tag_name	pi_tag_type	formula
101	F-01	Feed_Temp	PI_FT_101	pi	NULL
102	F-01	Calc_Eff	NULL	calculated	(T1 + T2) / 2
103	F-01	Max_Limit	NULL	constant	1200

Configuration Injection Utility

The `load_config_from_db` utility fetches operational parameters and injects them into Kedro nodes as `params`, ensuring the Python logic remains generic.

```
def load_config_from_db(case_id):
    # Fetches all relevant parameters for a specific furnace
    return {
        "tags": get_tags(case_id),
        "constraints": get_constraints(case_id),
```

```
        "objective": get_objective(case_id),
        "imputation": get_imputation(case_id),
        "pipeline": get_pipeline_config(case_id)
    }
```



---

5. Advanced Dependency Management: DAGs and Topological Sorting

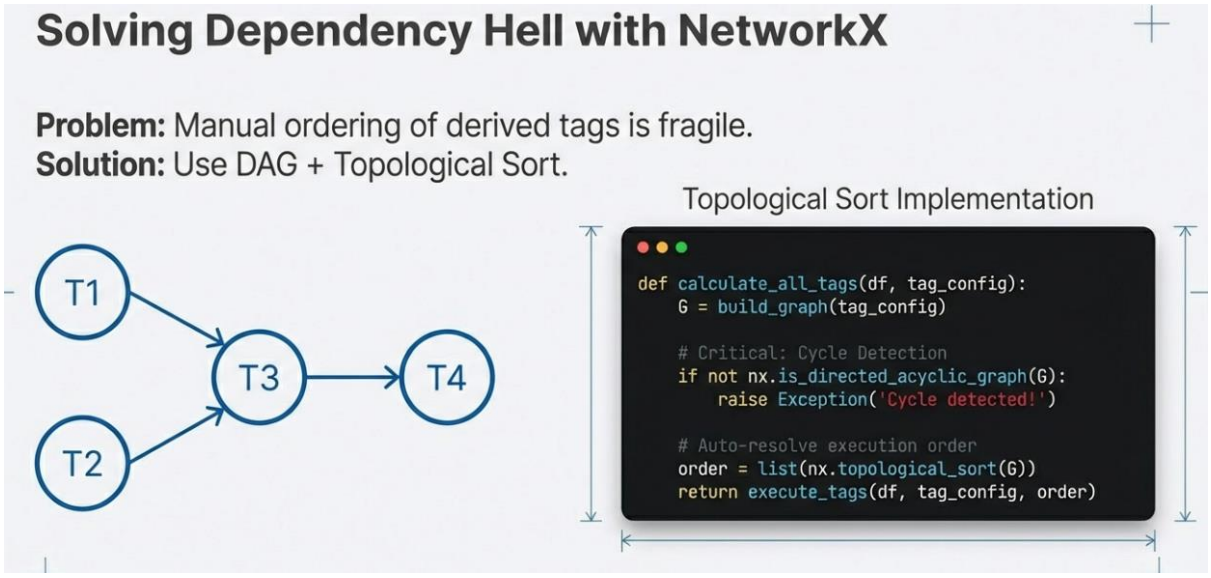
To resolve the "Dependency Trap" where manual tag ordering (e.g., calculating T3 before T4) leads to maintenance bottlenecks, the system employs automated graph theory.

Comparison of Approaches

- **Manual Ordering (Current):** Requires developers to define calculation sequences. It is error-prone, breaks when new tags are added, and leads to incorrect mathematical results.
- **Automated Sorting (Proposed):** Uses a **Directed Acyclic Graph (DAG)** via the `networkx` module to automatically determine the correct mathematical order based on defined dependencies.

Validation: Detecting Circular Dependencies

The system performs a critical validation step using `nx.is_directed_acyclic_graph(G)`.
The "So What?": A circular dependency (e.g., T3=T4+1, T4=T3+1) creates an infinite loop. Without this automated check, the orchestration engine would crash or hang indefinitely.



---

6. Engineering Excellence: Kedro Node Design Principles

High-quality furnace optimization requires "Single Responsibility" and "Pure Functions."

Best Practices vs. Anti-Patterns

Category	Bad Example (Anti-Pattern)	Good Design (Best Practice)
Logic Layer	**Monolithic:** One function connects to PI, fetches data, and cleans it.	**Atomic:** Nodes like `create_pi_connection`, `fetch_pi_data`, and `standardize_columns`.
Function Type	**Impure:** Modifies an external dataframe directly.	**Pure:** Takes input, uses `df.copy()`, and returns a new output.
Config Use	**Hardcoded:** `df["T1"].clip(0, 1200)`	**Config Injection:** Iterating through `config.items()` to apply rules dynamically.

Config Injection: Code Comparison

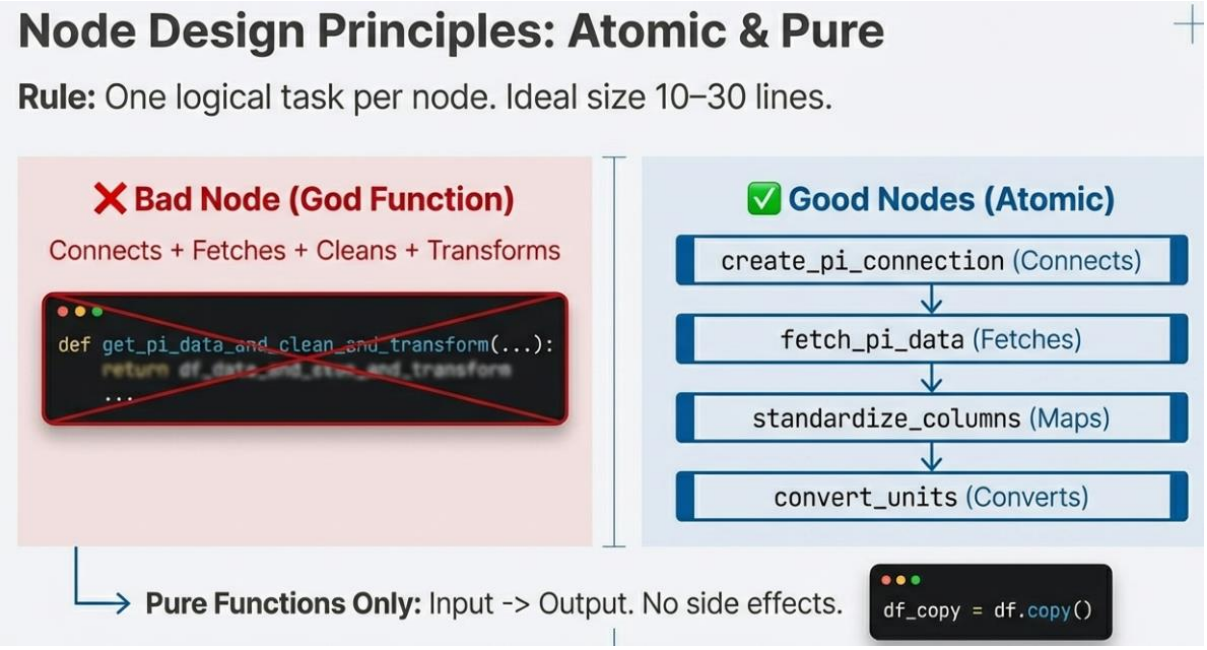
✕ Bad (Hardcoded)

```
def apply_limits(df):
    df["T1"] = df["T1"].clip(0, 1200)
    return df
```

✓ Good (Generic)

```
def apply_limits(df, config):
    df_copy = df.copy() # Pure Function
    for tag, rules in config.items():
        df_copy[tag] = df_copy[tag].clip(rules["min"], rules["max"])
    return df_copy
```

The Golden Rule: Nodes should be concise (**10–30 lines**), contain zero furnace-specific logic, and have no side effects.



---

7. Dynamic Engines: Formulas, Imputation, and Constraints

The framework uses "Engines" where Python acts as a runner for logic defined in the configuration.

- **Formula Engine:** Uses `df.eval()` to parse calculations stored in the DB. This avoids embedding math in Python scripts.
- **Constraint Engine:** Uses `df.query()` to evaluate boundary conditions dynamically.
- **Imputation Strategy:** Supports diverse methods defined in configuration:

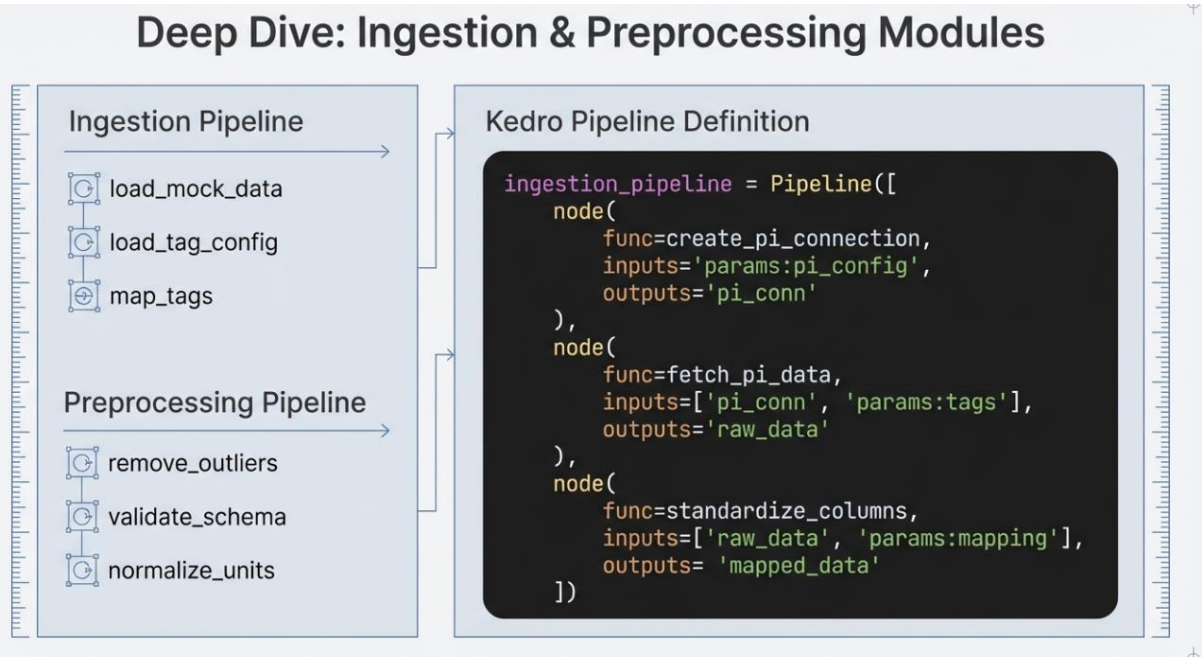
Method	Description
Clip	Bounds values within a min/max range.
Forward Fill	Carries the last known good value forward.
Interpolation	Smoothes gaps between data points.
Model-based	Uses ML logic to predict missing values.

---

8. Implementation Standards: Structure and Safety

Kedro Standard Folder Structure

```
project_root/
├── src/
│   ├── pipelines/
│   │   ├── macro_pipelines/
│   │   │   └── furnace_optimization/
│   │   │       ├── pipeline.py        # Orchestrates flow
│   │   │       └── runner.py          # Execution entry point
│   │   ├── micro_pipelines/
│   │   │   ├── ingestion/             # nodes.py, pipeline.py
│   │   │   ├── preprocessing/         # cleaning, validation
│   │   │   ├── feature_engineering/   # inferred tags (DAG)
│   │   │   ├── imputation/            # missing value handling
│   │   │   └── optimization/          # optimizer logic
│   ├── config/
│   │   └── db_loader.py               # Fetches configs from DB
│   ├── data_access/
│   │   ├── pi_client.py               # PI historian connection
│   │   └── db_client.py
│   ├── orchestration/
│   │   ├── pipeline_builder.py
│   │   └── dag_manager.py             # NetworkX logic
```



Critical Anti-Patterns and Business Impact

Anti-Pattern	Negative Business Impact
Hardcoded Tags	System cannot scale to new furnaces; high technical debt.
Tight Coupling	Components cannot be reused; requires full rewrites for new sites.
Mixed Logic Layers	Impossible to debug or test effectively; high downtime.

Testing Strategy

Level	What to Test
Unit Testing	Validation of each individual node (Pure function integrity).

Integration Testing	Verification of the end-to-end pipeline flow across furnace types.
Config Validation	Ensuring DB/YAML parameters meet required schemas before execution.

Conclusion: The Golden Rules Summary

To ensure the longevity of the framework, all development must adhere to these final principles: **Configuration drives everything**, **Nodes are atomic**, and **Pipelines are composable**. Logic should never be furnace-specific, and every component must be independently testable.